CS6905 - Cloud Information Management Systems

Project Design Submission

Student Information

Ali Rizvi #3802911

Fahim Faisal #3809190

Refat Ishrak Hemel #3810002

Date: March 15, 2026

Project Title

Concept Drift-Aware Cloud ML Pipeline for Smart Inventory Management

1. Introduction and Summary

1.1 Project Overview

The incapacity of conventional machine learning models to adjust to changing business patterns is a significant problem that contemporary inventory management systems must deal with (Lu et al., 2018). Using serverless cloud architecture and adaptive machine learning, our solution- a drift-aware cloud ML pipeline for smart inventory management creates a self-improving demand forecasting system that continuously learns from new data while optimizing operating costs.

1.2 Problem Context and Motivation

Today, most businesses rely on manual inventory rules or static forecasting models, which can result in two expensive outcomes: stockouts that cause lost sales or overstocking that tie up capital. Conventional ML deployments take a "train-once-and-forget" approach, even though machine learning offers improvements over conventional methods. These models function well at first, but they gradually deteriorate when consumer behavior changes because of market dynamics, seasonal variations, or advertising campaigns (Gama et al., 2014). Because the infrastructure needed for regular retraining is either unavailable or too costly to maintain, organizations hardly ever retrain models.

Traditional always-on server architectures also result in needless expenses. Although most businesses only need updated predictions once a day or once a week, inventory forecasting is by its very nature periodic. However, traditional deployments keep computing resources running around the clock, covering idle capacity during non-prediction periods (Adzic & Chatley, 2017).

1.3 Design Phase Objective

The design phase of this project focuses on architecting a serverless, event-driven ML system on AWS that achieves three core objectives:

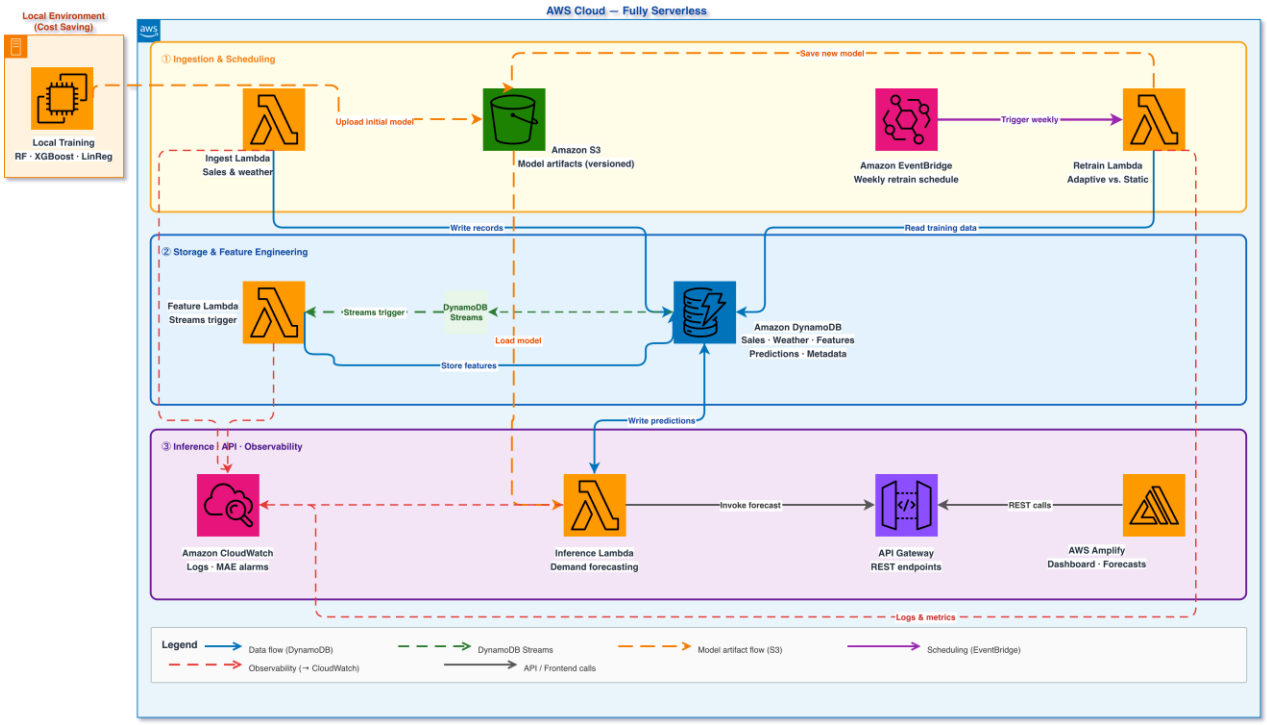
1. **Adaptive Learning Architecture**: Design a pipeline that automatically detects concept drift in sales patterns and triggers model retraining using recent data windows, ensuring predictions remain accurate as business conditions evolve.

2. **Cost-Optimized Serverless Infrastructure**: Architect a fully serverless solution using AWS Lambda, DynamoDB, and S3 that eliminates idle compute costs by executing only when triggered by scheduled events or new data arrivals.

3. **Scalable NoSQL Data Management**: Using DynamoDB, create a schema-flexible data model that supports a variety of data sources, such as sales transactions, weather observations, computed features, prediction outputs, and model metadata, without necessitating schema migrations during iterative development.

2. System Architecture

2.1 High-Level Architecture



Our architecture breaks down into three layers. Everything runs on **AWS serverless services**.

Layer 1 — Ingestion and Scheduling Sales and weather data come into the system through an **Ingest Lambda**. In a real deployment this would connect to point-of-sale

systems or external APIs, but for our project we simulate it by pushing weekly batches that represent different time periods — regular sales, a Christmas rush, winter clearance, and so on. **Amazon EventBridge** is responsible for the scheduling side of things. It fires on a weekly cadence to kick off retraining, and it can chain together multi-step workflows like running a validation check right after a new model is trained.

Layer 2 — Storage and Feature Engineering DynamoDB is the central database for the whole pipeline. Sales records, weather data, computed features, predictions, model metadata — it all goes into one DynamoDB table. We go into more detail on why we chose this design in Section 3. The important thing architecturally is **DynamoDB Streams**: whenever new records get written to the table, Streams picks up the change and automatically triggers a **Feature Engineering Lambda**. That Lambda computes things like 7-day rolling averages, lag features, and temporal signals (day of week, month, etc.), then writes those features back to DynamoDB so they're available for inference and retraining. Model artifacts — the actual serialized model files — live in **S3** with versioning turned on, which means we can always roll back if a retrained model turns out to be worse.

Layer 3 — Inference, API, and Observability When it's time to make predictions, the **Inference Lambda** pulls the latest model from S3, grabs the most recent features from DynamoDB, runs the forecast, and writes the predictions back to DynamoDB. Those predictions get exposed through a **REST API via API Gateway**, which is what the dashboard talks to. For monitoring, **CloudWatch** collects logs and metrics from every Lambda invocation, and we also push custom metrics like the per-cycle MAE so we can set up alarms if accuracy starts dropping. The dashboard itself is hosted on **AWS Amplify**, and it's where you can see everything at a glance — current predictions, inventory levels, and the comparison between the static and adaptive models.

2.2 Data Flow

Here's how data moves through the system end to end:

1. **Ingestion:** New sales and weather records get written to DynamoDB through the Ingest Lambda.
2. **Feature computation:** DynamoDB Streams picks up the new records and fires the Feature Lambda. It computes lag features, rolling averages, and encoded categoricals, then writes the results back to DynamoDB.
3. **Retraining:** On a weekly schedule, EventBridge triggers the Retrain Lambda. It pulls all the historical sales and feature data from DynamoDB, trains a fresh ensemble model (Random Forest + XGBoost + Linear Regression), checks it against the static baseline, and saves the new artifact to S3. Model metadata and accuracy metrics go back to DynamoDB.
4. **Inference:** The Inference Lambda — triggered either through the API or on a schedule — loads the current model from S3, reads features from DynamoDB, generates forecasts, and writes predictions back to DynamoDB.

5. **Monitoring:** CloudWatch captures execution logs, latency numbers, and our custom accuracy metrics. If MAE goes above a threshold we've configured, an alarm fires.

6. **Dashboard:** The Amplify frontend polls the API Gateway endpoint and renders charts for predictions, inventory, and the static-vs-adaptive MAE comparison.

2.3 Third-Party Integrations

- **Kaggle Dataset:** We're using a publicly available Retail Store Inventory Forecasting dataset from Kaggle with over 73,000 transactions. It includes features like store ID, product ID, category, region, price, discount, weather conditions, holiday/promotion flags, competitor pricing, and seasonality.

- **Local Training:** The first round of model training happens locally with scikit-learn, XGBoost, and pandas. Once the model is trained and validated, we serialize it and upload it to S3 as the starting point for the cloud pipeline.

3. Data Model Design

3.1 Why NoSQL (DynamoDB)

We looked at both relational and NoSQL databases for this project and ultimately went with DynamoDB. Here's why it made more sense for what we're building:

- **Schema differences across record types:** Our pipeline deals with five very different kinds of data, sales transactions, weather observations, computed features, prediction results, and model metadata. These don't look like each other. A sales record has things like `Units_Sold`, `Price`, `Discount`, and `Weather_Condition`. A model metadata record has `MAE`, `feature_list`, `training_window`, and `hyperparameters`. In a relational database, each of those would need its own table with a fixed schema, and every time we tweak a feature or add a new data source during development, we'd have to deal with schema migrations. DynamoDB sidesteps fixed schemas entirely because items in the same table can have totally different attributes. All five record types sit in one table without any upfront schema definition, which is helpful during iterative ML work where the feature set keeps changing.

- **Event-driven behavior:** DynamoDB Streams gives us event-driven behavior out of the box. When a new sales record hits the table, Streams captures it and triggers our Feature Lambda automatically. There's no polling, no external message queue, no change-data-capture tool to set up. If we were using a relational database like RDS, getting the same reactive behavior would mean bringing in something like Debezium with Kafka, or writing custom polling logic.

- **Serverless cost model:** With on-demand capacity, it scales up when we're ingesting a big batch of data and scales back down when nothing's happening. We only pay for

what we use. Relational options like RDS or Aurora need provisioned instances that run all the time whether there's traffic or not, which doesn't match our workload at all. Forecasting is bursty, we load data periodically, run predictions, and then the system sits idle until the next cycle.

- **Access patterns:** Every query in the system boils down to key-value lookups. We never need to join across unrelated tables or complex analytical queries. DynamoDB is built for exactly this kind of access and handles it at single-digit millisecond latency no matter how big the table gets.
- **Time-series structure:** Everything is partitioned by store and ordered by time, which maps perfectly to DynamoDB's partition key and sort key model. You'd need to set up explicit partitioning and indexing in a relational database to get the same query performance on time-range lookups.

3.2 Single-Table Design

We went with a single-table design, which is the standard recommended pattern for DynamoDB applications.

Primary Key Structure:

Partition Key (PK)	Sort Key (SK)	Record Type	Key Attributes
STORE#<store_id>	SALE#<date>#<product_id>	Sales transaction	Units_Sold, Price, Discount, Weather_Condition, Category, Region, Inventory_Level
STORE#<store_id>	FEATURE#<year>-W<week>	Computed features	Rolling_7, Sales_Lag_1, Sales_Lag_7, DayOfWeek, Month, encoded categoricals
STORE#<store_id>	PRED#<year>-W<week>	Demand prediction	Predicted_Units, Model_Version, Confidence
MODEL#<version>	META	Model metadata	MAE, RMSE, Hyperparameters, Feature_List, Training_Window, Created_At
MODEL#<version>	METRIC#<year>-W<week>	Weekly model metrics	Static_MAE, Adaptive_MAE, Improvement_Delta

Global Secondary Index (GSI):

GSI1 Partition Key	GSI1 Sort Key	Use Case
REGION#<region>	SALE#<date>	Pulls all sales in a given region for a date range without scanning every store's partition

The reason we choose a single table is basically due to DynamoDB streams, locking down with roles. To elaborate, one table means one DynamoDB Stream, which can trigger all our downstream Lambdas. If we split things across multiple tables, we'd have to wire up and manage separate streams for each one. Then DynamoDB supports transactional writes within a single table, so we can write a prediction and update model metrics atomically in one call. And finally, from an IAM perspective, one table is simpler to lock down than five. Fewer permission boundaries mean less room for misconfiguration.

3.3 Indexing and Partitioning

Partition key choice: STORE#<store_id> spreads data across DynamoDB's internal partitions fairly evenly to avoid hot-partition issues.

Sort key for range queries: Because the sort key includes both the record type prefix and a timestamp, we can do efficient range queries (e.g., getting all sales for a specific month).

On-demand capacity: Automatically adjusts to handle spikes during batch ingestion and costs basically nothing during idle periods.

TTL for cleanup: Prediction records older than 90 days are set to automatically expire, keeping the table size optimized.

4. Security and Scalability Considerations

4.1 Security

Encryption: DynamoDB encrypts all data at rest by default using AWS-managed keys (AES-256), so we don't need to configure anything extra there. S3 model artifacts get server-side encryption as well (SSE-S3). For data in transit, everything goes over TLS 1.2+, that covers Lambda talking to DynamoDB, Lambda reading from S3, API Gateway serving the dashboard, and all the internal AWS service-to-service communication.

Access Control: Every Lambda function gets its own IAM role, and we scope those roles down to only what each function actually needs. The Ingest Lambda can write to DynamoDB but can't touch S3 model artifacts. The Inference Lambda can read from both S3 and DynamoDB but can't delete or overwrite models. API Gateway endpoints use IAM authentication or API keys so random people can't hit our forecast endpoints. The S3 bucket has public access completely blocked. Only the specific Lambda execution roles are allowed to interact with it.

Network Security: All the AWS services talk to each other over AWS's internal network. We can also set up throttling and rate limiting on the API Gateway side to prevent anyone from hammering the public endpoints.

Auditing: CloudTrail logs every API call across the account, giving us a full record of who did what and when. CloudWatch Logs captures each Lambda invocation in detail, which makes it easy to trace data from the moment it enters the system all the way through to a prediction.

4.2 Scalability

Lambda: Lambda scales horizontally on its own. If a big batch of data comes in and the DynamoDB Stream has a lot of events to process, AWS spins up however many concurrent Lambda instances it needs. We don't have to plan capacity or configure any auto-scaling rules. The default regional concurrency limit of 1,000 is way more than enough for our use case.

DynamoDB: On-demand mode means DynamoDB adjusts its read and write throughput automatically based on traffic. A sudden burst of thousands of records during batch ingestion doesn't require any pre-warming or manual scaling. And because we partition by store ID, adding more stores just spreads the data across more partitions naturally.

S3: S3 is effectively unlimited storage. Our model artifacts are small. The serialized ensemble is typically under 50 MB, so storage costs are negligible. Versioning means every model we've ever trained is still available for rollback, and we didn't have to build anything custom for that.

5. Timeline and Risk Assessment

5.1 Project Timeline

This project follows a milestone-based timeline that reflects the major stages of system design and development:

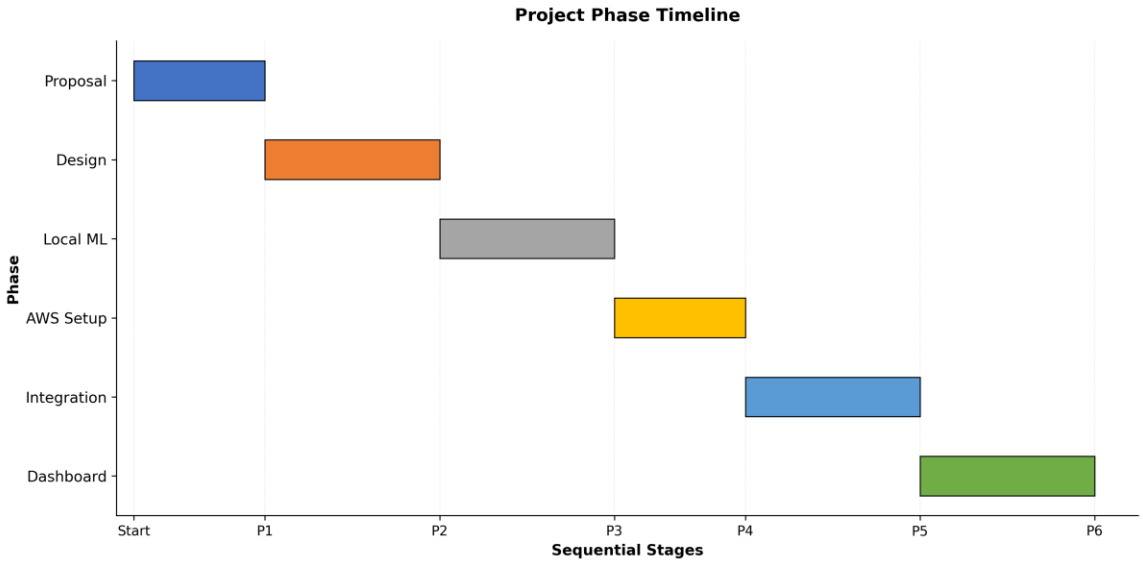


Figure 1: Gantt chart showing the six project phases and their sequential relationship

- **Milestone 1: Proposal.** This phase establishes the project foundation by identifying the concept drift challenge in inventory forecasting and proposing a serverless cloud ML solution using AWS services

- **Milestone 2: Design.** This phase focuses on architecting the complete system including data flow design, DynamoDB table schemas, API specifications, security configurations, and scalability strategies. The design document produced in this phase serves as the blueprint for implementation.

- **Milestone 3: Local ML.** This phase develops and validates the machine learning approach locally. The ensemble model (Random Forest + XGBoost + Linear Regression) is trained and tested with both static and adaptive retraining strategies to establish baseline performance metrics before cloud deployment.

- **Milestone 4: AWS Setup.** This phase deploys the AWS infrastructure including Lambda functions, DynamoDB tables, S3 buckets, API Gateway endpoints, EventBridge schedules, and CloudWatch monitoring. IAM roles and security policies are configured to enable secure inter-service communication.

- **Milestone 5: Integration.** This phase connects all components into a unified pipeline. Data ingestion, feature engineering, model training, inference, and monitoring are integrated through event-driven triggers using DynamoDB Streams and EventBridge schedules.

- **Milestone 6: Dashboard.** This final phase builds the AWS Amplify frontend dashboard for visualizing forecasts, inventory states, and retraining history. A demonstration using seasonal data variations validates that the adaptive retraining approach successfully handles concept drift.

5.2 Risk Assessment and Mitigation Strategies

The project presents a small set of important risks. The table below summarizes the major risks and the proposed mitigation strategy for each one.

Risk	Likelihood	Impact	Mitigation Strategy
Lambda cold starts	Medium	Low	Use provisioned concurrency if dashboard latency becomes an issue.
Retraining timeout	Medium	High	Trim training window or move to AWS Step Functions.
DynamoDB throttling	Low	Medium	On-demand mode already handles most bursts; add exponential backoff and batch write retry logic in the Ingest Lambda if needed
Concept drift invisible	Medium	High	Dramatize seasonal shifts in simulation data for the demo.
Worse retrained model	Low	Medium	Implement a validation gate comparing MAE before deployment.

References

Adzic, G., & Chatley, R. (2017). Serverless computing: Economic and architectural impact. In *Proceedings of the 2017 11th Joint Meeting on Foundations of Software Engineering* (pp. 884–889). https://doi.org/10.1145/3106237.3117767

DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). Dynamo: Amazon's highly available key-value store. *ACM SIGOPS Operating Systems Review*, *41*(6), 205–220. https://doi.org/10.1145/1323293.1294281

Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM Computing Surveys*, *46*(4), 1–37. https://doi.org/10.1145/2523813

Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE Transactions on Knowledge and Data Engineering*, *31*(12), 2346–2363. https://doi.org/10.1109/TKDE.2018.2876857